第4章 数据可视化包Matplotlib

- 4.1 Matplotlib绘图基础
- 4.2 Matplotlib常用图形绘制

4.1 Matplotlib绘图基础

- Matplotlib是Python中一个二维绘图包，能够非常简单的实现数据可视化。Matplotlib最早由John Hunter于2002年启动开发，其目的是为了构建一个Matlab式的绘图函数接口。下面详细介绍Matplotlib图像构成、Matplotlib图像基本绘图流程、中文字符显示、坐标轴字符刻度标注等基本绘图知识。

4.1.1 Matplotlib图像构成

- Matplotlib图像大致可以分为如下4个层次结构：

1. canvas（画板）

位于最底层，导入matplotlib库时就自动存在。

2. figure（画布）

建立在canvas之上，从这一层就能开始设置其参数。

3.axes（子图）

将figure分成不同块，实现分面绘图。

4.图表信息（构图元素）

添加或修改axes上的图形信息，优化图表的显示效果。

4.1.1 Matplotlib图像构成

- 为了方便快速绘图，Matplotlib通过pyplot模块提供了一套和Matlab类似的命令API,将众多绘图对象所构成的复杂结构隐藏在这套API内部，这些对象对应图形的一个个图形元素（如坐标轴、曲线、文字等），模块给每个对象分配函数，以此对该图形元素进行操作，而不影响其他部分。创建好画布后，只需调用pyplot模块所提供的函数，仅几行代码就可以实现添加、修改图形元素或在原有图形上绘制新图形。

在Anaconda发行版中已经集成了Matplotlib库，直接导入pyplot模块就可以使用了。导入方法为：import matplotlib.pyplot as plt，如图4-1所示。

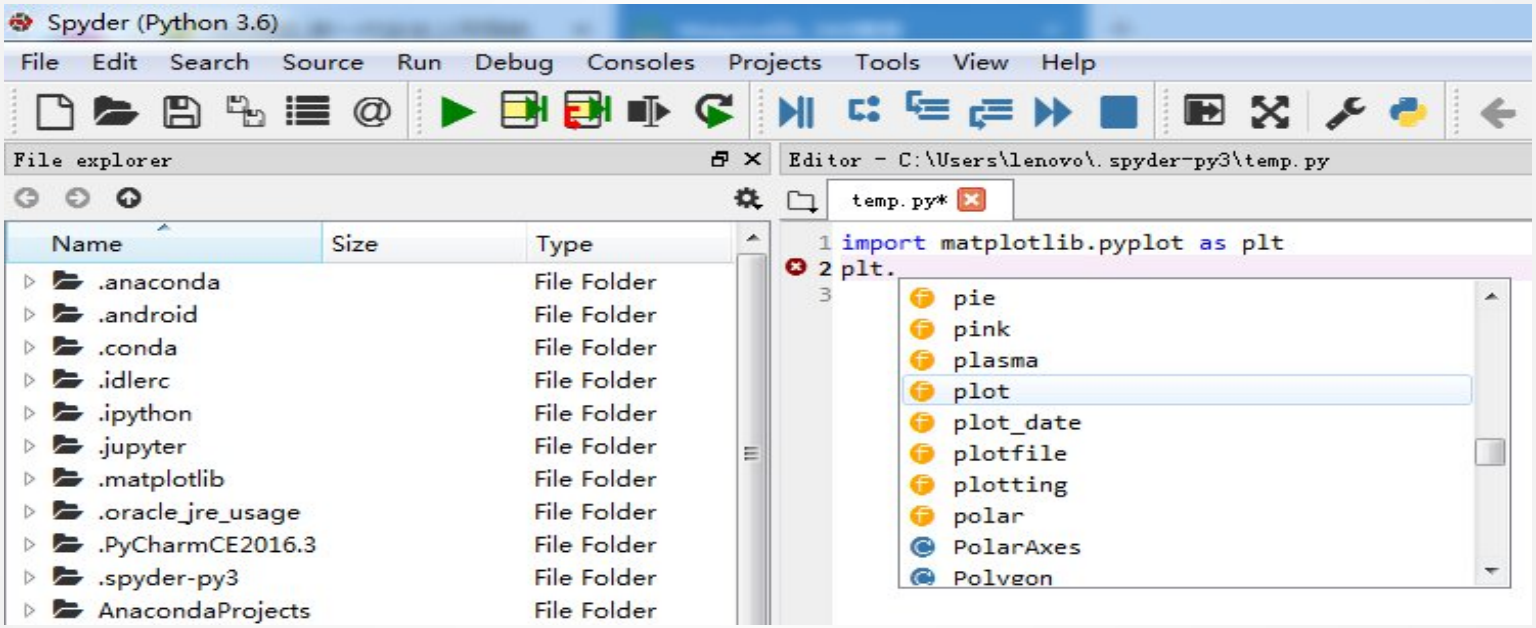

图4-1

图4-1显示了在temp.py脚本文件中导入了Matplotlib包的pyplot模块，并简称为plt。导入pyplot模块后就可以绘图了，利用pyplot模块绘图的基本流程如图4-2所示。

4.1.2 Matplotlib绘图基本流程

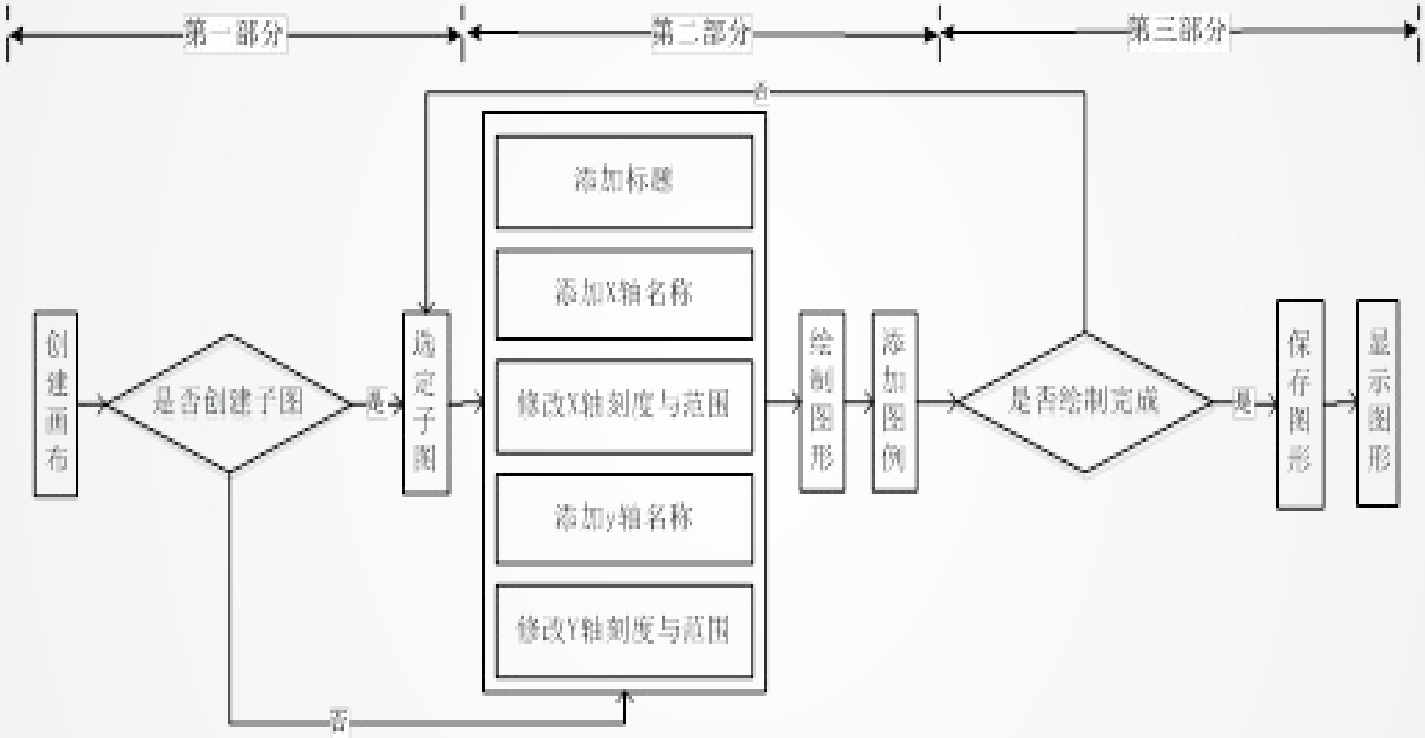

图 4-2

pyplot基本绘图流程

- 首先是创建画布与创建子图。第一部分主要是构建出一张空白的画布，如果需要同时展示几个图形，可将画布划分为多个部分。然后使用对象方法来完成其余的工作，示例代码如下：
- plt.figure(1)　　# 创建第一个画布
- plt.subplot(2, 1, 1)　　# 画布划分为2×1图形阵，选择第1张图片
- 其次是添加画布内容。第二部分是绘图的主体部分。添加标题、坐标轴名称等步骤与绘制图形是并列的，没有先后顺序，可以先绘制图形，也可以先添加各类标签，但是添加图例一定要在绘制图形之后。pyplot模块中添加各类标签和图例的函数如表4-1所示。

表 4-1 pyplot添加各类标签的常用函数

函数名称	函数作用
title	在当前图形中添加标题，可以指定标题的名称、位置、颜色、字体大小等参数
xlabel	在当前图形中添加x轴名称，可以指定位置、颜色、字体大小等参数
ylabel	在当前图形中添加y轴名称，可以指定位置、颜色、字体大小等参数
xlim	指定当前图形x轴的范围，只能确定一个数值区间，而无法使用字符串标识
ylim	指定当前图形y轴的范围，只能确定一个数值区间，而无法使用字符串标识
xticks	指定x轴刻度的数目与取值
yticks	指定y轴刻度的数目与取值
legend	指定当前图形的图例，可以指定图例的大小、位置、标签

4.1.2 Matplotlib绘图基本流程

最后是图形保存与展示。绘制图形之后，可使用matplotlib.pyplot.savefig()函数保存图片到指定路径，使用matplotlib.pyplot.show()函数展示图形。综合整体流程绘制函数“y=x^2”与“y=x”

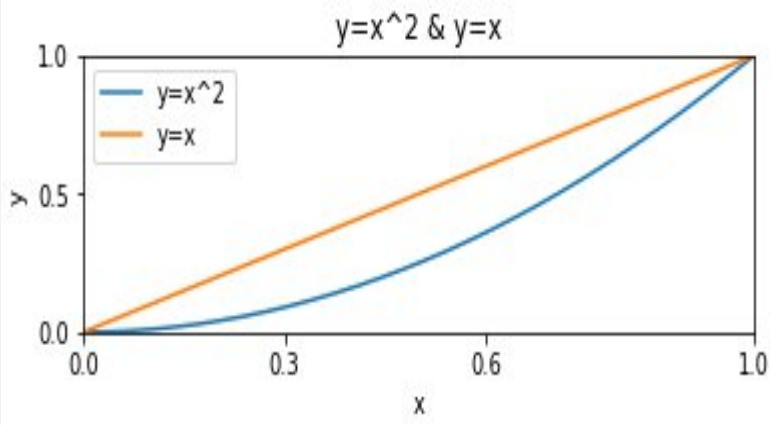

图4-3

图像示例代码如下：

```
import matplotlib.pyplot as plt
import numpy as np
plt.figure(1) # 创建画布
x = np.linspace(0, 1, 1000)
plt.subplot(2, 1, 1) # 分为2×1图形阵，选择第1张图片绘图
plt.title('y=x^2 & y=x') # 添加标题
plt.xlabel('x') # 添加x轴名称‘x’
plt.ylabel('y') # 添加y轴名称‘y’
plt.xlim((0, 1)) # 指定x轴范围（0,1）
plt.ylim((0, 1)) # 指定y轴范围（0,1）
plt.xticks([0, 0.3, 0.6, 1]) # 设置x轴刻度
plt.yticks([0, 0.5, 1]) # 设置y轴刻度
plt.plot(x, x ** 2)
plt.plot(x, x)
plt.legend(['y=x^2', 'y=x']) # 添加图例
plt.savefig('1.png') # 保存图片
plt.show()
```

执行结果如图4-3所示。

4.1.3 中文字符显示

值得注意的是默认的pyplot字体并不支持中文字符的显示，因此需要通过修改font.sans-serif参数来修改绘图时的字体，使得图形可以正常显示中文。同时，由于修改字体后，会导致坐标轴中负号“-”无法正常显示，因此需要同时修改axes.unicode_minus参数。

- 示例代码如下：
- import numpy as np
- import matplotlib.pyplot as plt
- x = np.arange(0, 10, 0.2)
- y = np.sin(x)
- plt.title('sin曲线')
- plt.plot(x, y)
- plt.savefig('2.png')
- plt.show()

执行结果如图4-4所示。

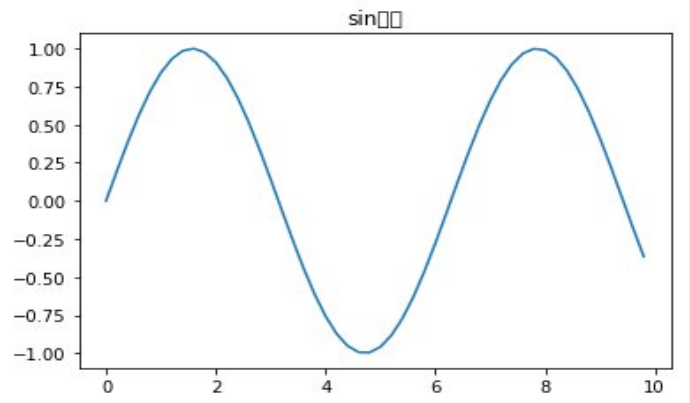

图4-4

4.1.3 中文字符显示

- 从图4-4可以看出，中文字符没有能显示出来。同时应该注意到在示例代码中，并没有创建画布的命令，实际上只要调用了绘图命令系统就会默认创建一个画布，并在该画布上绘图。为了显示中文字符可以对示例代码修改如下：

```
import numpy as np
import matplotlib.pyplot as plt
x = np.arange(0, 10, 0.2)
y = np.sin(x)
plt.rcParams['font.sans-serif'] = 'SimHei'  # 设置字体为SimHei
plt.rcParams['axes.unicode_minus'] = False  # 解决负号“-”显示异常
plt.title('sin曲线')
plt.plot(x, y)
plt.savefig('2.png')
plt.show()
```

修改后的程序代码执行结果如图4-5所示。

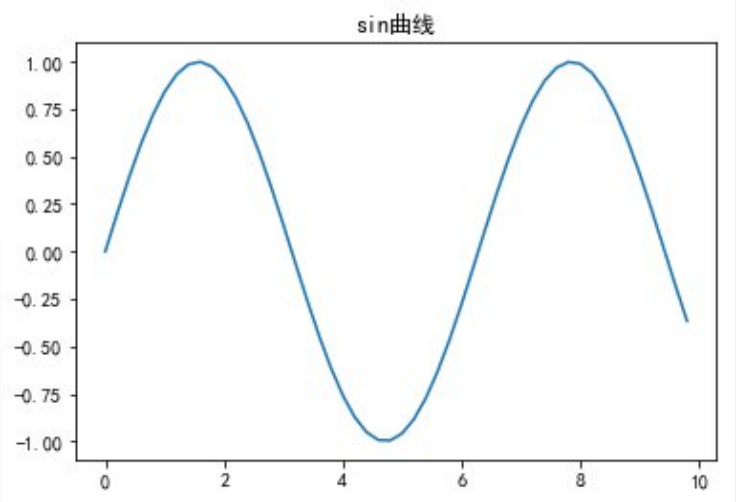

图4-5

4.1.4 坐标轴字符刻度标注

绘图过程中还有一个关键的问题就是坐标轴的字符刻度表示问题，比如绘制2018年-2019年某产品各个季度的销售额走势图，两年各季度的销售数据依次为：100、104、106、95、103、105、115、100（单位：万元）。一般的绘图代码如下：

图4-6

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1,2,3,4,5,6,7,8])
#季度标号
y =
np.array([100,104,106,95,103,105,115,100]) #销售额
plt.rcParams['font.sans-serif'] =
'SimHei'    # 设置字体为SimHei
plt.title('某产品2018-2019各季度销售额')
plt.plot(x, y)
plt.xlabel('季度标号')
plt.ylabel('销售额（万元）')
plt.show()
```

执行结果如图4-6所示。

4.1.4 坐标轴字符刻度标注

从图4-6可以看出，横轴的意义没有突显出来，造成了图像的可读性比较差。实际上可以用xxx年xxx季度来表示，这样图像的可读性就更强。对横轴进行字符刻度标注可以通过表4-1中的xticks函数来实现。示例代码如下：

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1,2,3,4,5,6,7,8])
#季度标号
y =
np.array([100,104,106,95,103,105,115,100]) #销售额
v=['2018年一季度','2018年二季度','2018年三季度','2018年四季度',
   '2019年一季度','2019年二季度','2019年三季度','2019年四季度']
plt.rcParams['font.sans-serif'] = 'SimHei'     # 设置字体为SimHei
plt.title('某产品2018-2019各季度销售额')
plt.plot(x, y)
plt.xlabel('季度')
plt.xticks(x, v, rotation = 45) #v为与x对应的字符刻度，rotation为旋转角度
plt.ylabel('销售额（万元）')
plt.show()
```

执行结果如图4-7所示。

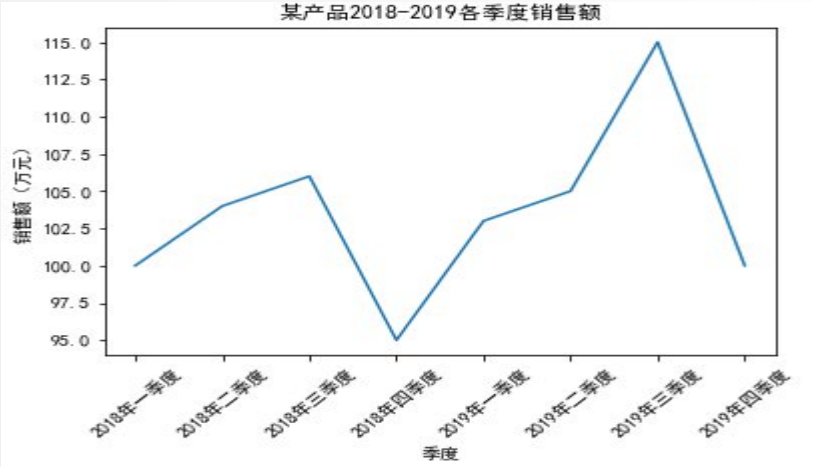

图4-7